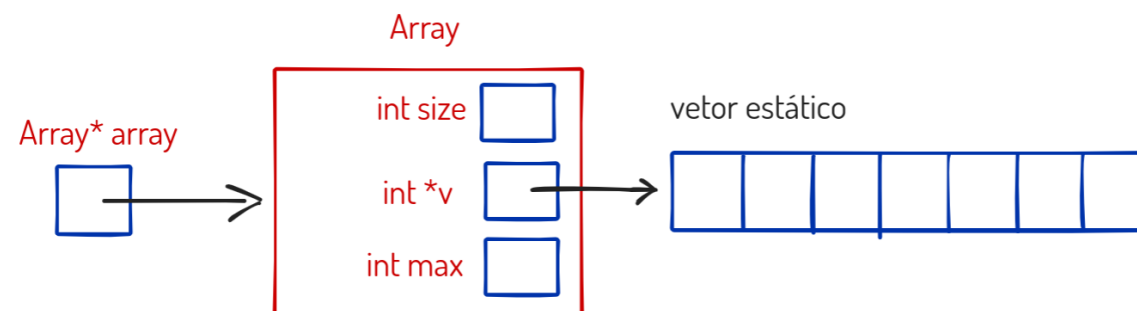


Algoritmos e Lógica de Programação - ALP

Prof. Dr. Murillo Carvalho



Algoritmos e Lógica de Programação



AULA VETORES DINÂMICOS

<https://www.raylib.com/>

Algoritmos e Lógica de Programação

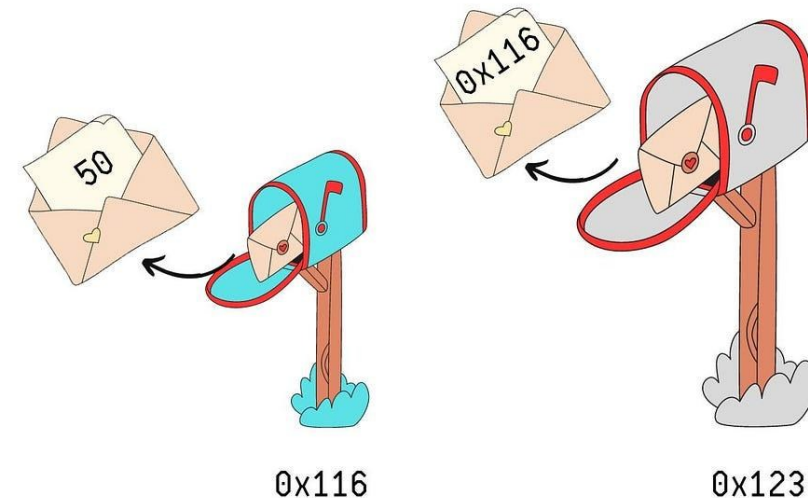
O que é um ponteiro?

Um **ponteiro** é uma variável que **guarda o endereço de outra variável**.

Imagine assim:

- Sua casa = valor (ex: número 10)
- Endereço da casa = localização na memória
- Ponteiro = um papel onde está escrito esse endereço

ponteiro não guarda o valor diretamente
Ele guarda “onde está” o valor.



Algoritmos e Lógica de Programação

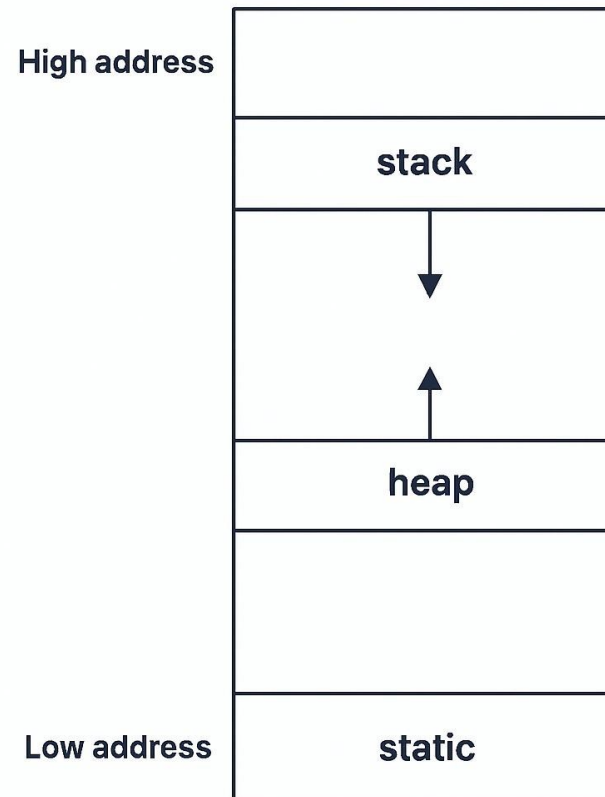
Exemplo simples:

```
int x = 10;  
int *p = &x;
```

- `x` guarda o valor 10
- `&x` = endereço de `x`
- `p` = ponteiro que aponta para `x`

Algoritmos e Lógica de Programação

O que é malloc?



Algoritmos e Lógica de Programação

malloc

significa **memory allocation** (alocação de memória).

Ele serve para **pedir um espaço na memória durante o programa**.

Analogia:

- É como **alugar um espaço vazio** em um depósito.

Exemplo:

```
int *p = (int *) malloc(sizeof(int));
```

- Você pediu espaço para **1 inteiro**
- `p` recebe o endereço desse espaço

Algoritmos e Lógica de Programação

O que é calloc / realloc?

calloc

- Igual ao malloc, mas **já zera a memória**

```
int *p = (int *) calloc(5, sizeof(int));
```

Cria 5 posições já com valor 0

Algoritmos e Lógica de Programação

realloc

- Serve para **aumentar ou diminuir um espaço já criado**

Analogia:

- Você alugou um depósito pequeno → agora quer maior

```
p = (int *) realloc(p, 10 * sizeof(int));
```

Algoritmos e Lógica de Programação

O que é free?

`free` serve para devolver a memória que você alugou.

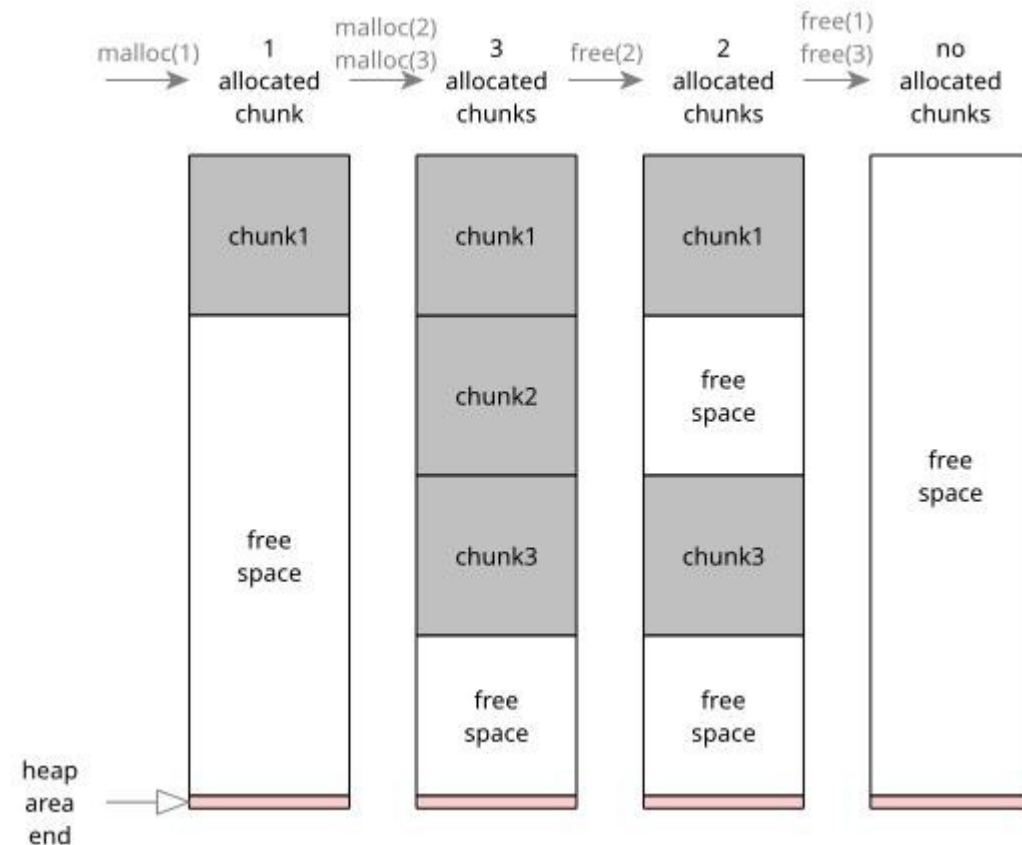
Analogia:

- Você alugou um espaço → precisa devolver depois

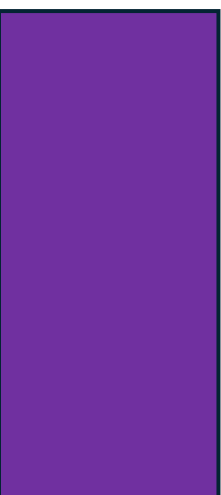
`free(p);`

Se não usar `free`, ocorre:

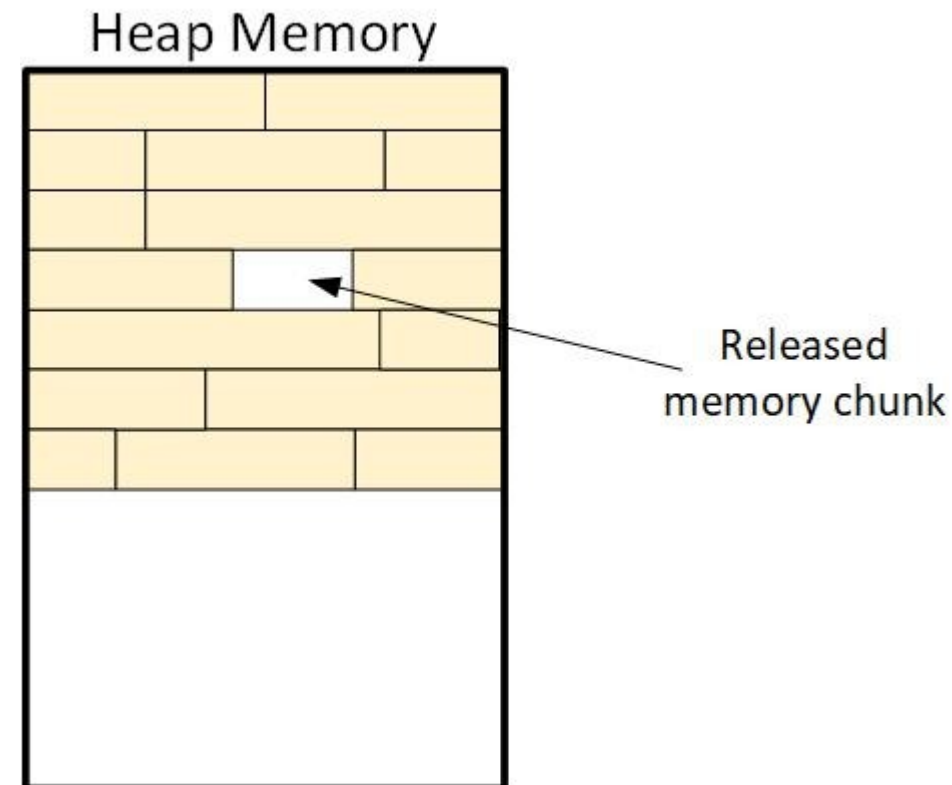
✗ vazamento de memória (memory leak)



Algoritmos e Lógica de Programação



Memory Chunk = pedaço de memória

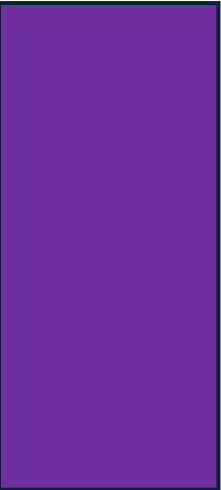


Algoritmos e Lógica de Programação

Resumo

- **Ponteiro** → guarda endereço
- **malloc** → pede memória
- **calloc** → pede memória já zerada
- **realloc** → muda o tamanho da memória
- **free** → devolve a memória

Algoritmos e Lógica de Programação



DECORE E APRENDAM!!

“Ponteiro aponta, malloc pega, realloc ajusta e free devolve.”

Algoritmos e Lógica de Programação

O que essa imagem representa?

Ela representa a **memória RAM** do processo do programa.

O programa divide a memória em áreas:

STACK → variáveis locais

HEAP → memória dinâmica
(malloc/realloc)

STATIC → variáveis
globais/estáticas



Algoritmos e Lógica de Programação

A RELAÇÃO COM VETOR DINÂMICO

Vetor normal (estático)

```
int v[5];
```

Vai geralmente para a **STACK**

Algoritmos e Lógica de Programação

Vetor dinâmico

```
int *v = malloc(5 * sizeof(int));
```

A variável v fica na STACK

MAS os dados do vetor ficam na HEAP

Algoritmos e Lógica de Programação

O QUE É STACK?

A STACK é:

- rápida
- automática
- pequena

Ela guarda:

- variáveis locais
- chamadas de função

Ex:

```
int x = 10;
```

Algoritmos e Lógica de Programação

O QUE É HEAP?

A HEAP é:

- grande
- flexível
- manual

Ela guarda:

- memória dinâmica
- malloc/calloc/realloc

Algoritmos e Lógica de Programação

Por que usar HEAP?

Porque a STACK é limitada

.

Se fizer:

```
int v[100000000];
```



Pode dar:

STACK OVERFLOW

(estouro da pilha)

Algoritmos e Lógica de Programação

Solução

Usar HEAP:

```
int *v = malloc(100000000 * sizeof(int));
```

A HEAP suporta estruturas muito maiores.

Algoritmos e Lógica de Programação

O QUE O REALLOC FAZ?

O `realloc` atua na HEAP.

Ele:

- aumenta
 - diminui
 - move
- a área alocada.

Memory leak (ou vazamento de memória) é uma falha que ocorre quando um programa aloca um espaço na memória RAM para realizar uma tarefa, mas não a libera quando a operação é concluída.

E O FREE?

O `free` devolve espaço da HEAP.

Sem isso:

✳ memory leak

Algoritmos e Lógica de Programação

Vai para **STACK** quando...

Você cria variável “normal”, local da função.

Exemplo:

```
int x;  
int v[10];  
char nome[50];
```

Tudo isso vai para a **STACK**

Porque:

- tamanho conhecido
- criado automaticamente
- destruído automaticamente

Algoritmos e Lógica de Programação

Vai para HEAP quando...

Você usa:

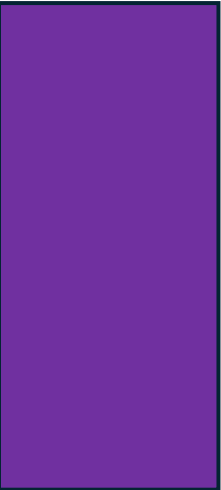
- malloc
- calloc
- realloc

Exemplo:

```
int *v = malloc(10 * sizeof(int));
```

O espaço do vetor vai para a **HEAP**

Algoritmos e Lógica de Programação



REGRA !!!!!

Sem malloc?

→ STACK

Com malloc?

→ HEAP

Algoritmos e Lógica de Programação

EXEMPLO MUITO IMPORTANTE STACK

```
void teste() {  
    int x = 10;  
}
```

Quando a função termina:
x desaparece automaticamente

Algoritmos e Lógica de Programação

HEAP

```
void teste() {  
    int *p = malloc(sizeof(int));  
}
```

Quando a função termina:

A memória continua existindo
Por isso precisa:

```
free(p);
```

Algoritmos e Lógica de Programação

ATENÇÃO

**automaticamente vai para a STACK. Tudo que é criado
manualmente**

Tudo que é criado com malloc vai para a HEAP.

Algoritmos e Lógica de Programação

PARA FINALIZAR:

Um **vetor estático** (ou *array* estático) em C é uma estrutura de dados que armazena uma sequência de elementos de **tamanho fixo** e do mesmo tipo. Ele é chamado de "estático" porque o seu tamanho deve ser definido no código-fonte e não pode ser alterado durante a execução do programa

O **vetor dinâmico** em C é um bloco de memória cujo tamanho pode ser **definido ou modificado durante a execução do programa**.

Diferente do estático, ele não tem tamanho fixo na compilação. Você pode pedir ao sistema operacional exatamente a quantidade de memória que precisa enquanto o programa roda.